

AI FAQ ASSISTANT

Phase 7 :Project Testing Phase

7.1 Project Overview

- The AI FAQ Assistant is a secure REST API platform for FAQ creation, authentication and AI-powered content management.
- Technology stack: Node.js, Express.js, MongoDB, Mongoose and Google Gemini SDK.
- JWT is used for stateless authentication and route protection.
- bcrypt is used to hash user passwords.
- The system supports FAQ CRUD operations, category management and semantic/keyword search.

7.2 Testing Objectives

- Verify that every major API function behaves according to its requirements.
- Validate user registration and login flows.
- Verify JWT authentication and authorization for protected routes.
- Check FAQ create, read, update and delete operations.
- Validate search and AI-generated FAQ functionality.
- Confirm schema validation, error handling and data integrity.
- Identify defects before deployment.

7.3 Testing Scope

- Authentication: registration, login and token validation.
- FAQ management: create, read, update and delete.
- Search: keyword-based FAQ search.
- AI automation: generation of FAQ question, answer and category.
- Category organization and access control.
- Security: password hashing and protected endpoints.
- Error handling and invalid request va

7.4 Test Environment

- Operating systems: Windows 10/11, macOS or Linux.
- Node.js version 18 or above.
- npm version 9 or above.
- Express.js backend with MongoDB/Mongoose.
- Postman or ThunderClient for API testing.
- Visual Studio Code or similar code editor.
- Minimum hardware stated in the project: Intel Core i5 8th Gen / Ryzen 5 equivalent, 8 GB RAM and 1 GB available storage.

7.5 Testing Approach

- Functional testing: verifies API behavior against expected results.
- Integration testing: verifies interaction between routes, middleware, services and MongoDB.
- Security testing: verifies authentication, authorization and protected resources.
- Validation testing: checks required fields, data types and schema rules.
- Negative testing: checks invalid credentials, missing fields and unauthorized requests.
- API testing is performed through Postman/ThunderClient.

7.6 Authentication Test Cases

- TC-AUTH-01: Register with valid name, email and password → user should be created.
- TC-AUTH-02: Register with missing required data → request should be rejected.
- TC-AUTH-03: Register with duplicate email → duplicate account should not be created.
- TC-AUTH-04: Login with valid credentials → authentication response/token should be returned.
- TC-AUTH-05: Login with invalid password → access should be rejected.
- TC-AUTH-06: Access a protected endpoint without JWT → request should be rejected.

7.7 FAQ CRUD Test Cases

- TC-FAQ-01: Create FAQ using valid authenticated request → FAQ should be stored.
- TC-FAQ-02: Read published FAQ data → requested FAQ information should be returned.
- TC-FAQ-03: Update an FAQ owned by the authenticated creator → changes should be saved.
- TC-FAQ-04: Delete an owned FAQ → the FAQ should be removed.
- TC-FAQ-05: Attempt to modify another user's FAQ → authorization should prevent the operation.
- TC-FAQ-06: Submit FAQ without required question/answer/category → validation should reject it.

7.8 Search and Category Testing

- Verify GET /api/faqs/search?q=configure returns matching FAQ records.
- Test search with an existing keyword.
- Test search with a keyword that has no match.
- Verify category values are stored and returned correctly.
- Verify a category can contain multiple FAQ documents.
- Check that search results remain accessible according to the route's access rules.

7.9 AI Generation Testing

- Verify POST /api/ai/generate-faq works for an authenticated user.
- Send a valid topic and check that generated FAQ content is returned.
- Verify generated output includes topic-related question/answer information.
- Test missing or empty topic input and verify validation/error handling.
- Verify unauthorized users cannot access the protected AI generation route.
- Check that generated content can later be saved as an FAQ when the application workflow supports it.

7.10 Security Testing

- Verify JWT tokens are required for protected FAQ and AI routes.
- Verify invalid or expired authentication context is rejected.
- Verify passwords are stored using bcrypt hashing rather than plain text.
- Verify users can modify only their permitted resources.
- Verify public users cannot create, update or delete FAQ records.
- Verify centralized error handling does not expose sensitive database information.

7.11 Validation & Error Handling

- Verify Mongoose schema validation for required fields.
- Check invalid email or missing user fields where applicable.
- Check missing FAQ question, answer or category.
- Verify malformed request bodies are handled safely.
- Verify the centralized error middleware returns standardized API errors.
- Confirm database or application errors do not leak internal implementation details.

7.12 Sample API Test Matrix

Register: POST /api/auth/register – Public – Valid user details – Expected: account created.

Login: POST /api/auth/login – Public – Valid credentials – Expected: authentication/token response.

Create FAQ: POST /api/faqs – Private – Bearer JWT + FAQ JSON – Expected: FAQ created.

Search FAQ: GET /api/faqs/search?q=configure – Public – Keyword query – Expected: matching FAQs.

Generate FAQ: POST /api/ai/generate-faq – Private – Bearer JWT + topic – Expected: AI-generated FAQ pair.

The project document specifies these routes as the API testing demonstration endpoints.

7.13 Expected Testing Results

- All valid requests should produce the expected successful API response.
- Invalid inputs should be rejected through validation and error handling.
- Protected routes should be accessible only to authorized users.
- FAQ ownership rules should prevent unauthorized modifications.
- Search should return relevant matching FAQ documents.
- AI generation should process valid authenticated topics.
- Testing should confirm data integrity and secure API behavior.
- Any failed test case should be recorded as a defect and retested after correction.

7.14 Conclusion

Source basis: supplied AI FAQ Assistant project document.

- Testing ensures the AI FAQ Assistant meets its functional, security and data-integrity requirements.
- API testing with Postman/ThunderClient provides direct verification of backend endpoints.
- Authentication, CRUD, search, AI generation, validation and error handling are the major testing areas.
- Successful completion of these tests supports reliable operation of the backend.
- Project testing demonstration and demo links are included in the supplied project document

